Persistencia de datos

Persistencia y diseño mínimo de base de datos

1. Propósito de la práctica

Deberán decidir si su proyecto requiere persistencia de datos y, en caso necesario, proponer un diseño mínimo de base de datos o una alternativa de almacenamiento.

2. Objetivo de aprendizaje

Decidir cómo se manejarán los datos, verificando que la estructura generada esté alineada con los requerimientos del Sistema.

3. Entregable general

1. Decisión sobre persistencia de datos: base de datos, archivos, memoria temporal o salida en consola.
2. Justificación de la decisión de persistencia.
3. Diseño mínimo de base de datos, si aplica.
4. Módulo responsable del manejo de datos.
5. Bitácora de uso de IA.

4. Roles sugeridos

Rol	Responsabilidad
Revisor técnico / diseñador de datos	Decide si se usará base de datos o alternativa de almacenamiento. Revisa modularidad y coherencia técnica.
Tester / documentador	Verifica propuesta de persistencia sugerida, documenta resultados, actualiza README o bitácora y verifica evidencia.
Integrante adicional, si aplica	Apoya en estructura del proyecto, revisión de trazabilidad o integración documental.

5. Actividad paso a paso

Paso 1. Decidir la persistencia de datos

Antes de codificar el primer módulo, el equipo debe decidir si el proyecto necesita guardar información después de cerrar el programa.

Pregunta	Respuesta del equipo
¿El proyecto requiere guardar información después de cerrar el programa?	
¿Qué información se guardará?	
¿Se usará base de datos?	
¿Qué alternativa se usará si no hay base de datos?	
¿Qué tipo de base de datos se usará si aplica?	
¿Qué módulo administrará los datos?	
¿Qué requerimientos justifican esta decisión?	

Paso 2. Elegir una ruta de trabajo

Ruta A: proyectos que NO requieren base de datos

El equipo debe justificar por qué no necesita base de datos y qué alternativa usará.

- Archivo CSV, JSON, TXT o .log.
- Salida en consola.
- Reporte generado en PDF, HTML o Excel.
- Datos temporales en memoria.
- Archivo de configuración.

Elemento	Respuesta
Justificación de no usar base de datos	
Alternativa de almacenamiento	
Ejemplo de salida o archivo esperado	
Módulo responsable	
Requerimiento relacionado	

Ruta B: proyectos que SÍ requieren base de datos

El equipo debe proponer un diseño mínimo antes de escribir código relacionado con datos.

- Justificación del uso de base de datos.
- Tipo de base de datos: SQLite, MySQL, PostgreSQL, MongoDB u otra.
- Entidades o tablas principales.
- Atributos y tipos de datos.
- Llaves primarias y relaciones.
- Operaciones necesarias: crear, consultar, actualizar, eliminar.
- Módulo responsable de la persistencia.

Diseño mínimo de base de datos

Entidad / Tabla	Atributos principales	Llave primaria	Relaciones	Requerimiento relacionado

Diccionario de datos básico

Campo	Tipo de dato	Descripción	Obligatorio	Ejemplo

Operaciones de datos

Operación	Descripción	Módulo responsable
Crear		
Consultar		
Actualizar		
Eliminar		

6. Prompts específicos para persistencia y base de datos

Prompt para decidir si se requiere base de datos

```
Actúa como arquitecto de software y diseñador de datos.

Con base en nuestro proyecto, requerimientos, historias de usuario y diseño modular, analiza si necesitamos base de datos o si basta con archivos u otra forma de almacenamiento.

Responde:
1. Si el proyecto necesita persistencia.
2. Qué datos deberían guardarse.
3. Si conviene usar base de datos o archivos.
4. Qué tipo de base de datos recomendarías, si aplica.
5. Qué entidades o tablas serían necesarias.
6. Qué atributos tendría cada entidad.
7. Qué relaciones existirían.
8. Qué operaciones CRUD serían necesarias.
9. Qué módulo debería manejar el acceso a datos.
10. Qué riesgos o complejidades debemos evitar.

No generes código todavía. Primero ayúdanos a tomar la decisión de diseño.
```

Prompt para proyectos con base de datos

```
Actúa como diseñador de base de datos.

Nuestro proyecto requiere almacenar la siguiente información:
[DESCRIBIR DATOS]

Ayúdanos a proponer un diseño mínimo de base de datos.

Incluye:
1. Entidades o tablas.
2. Atributos.
3. Llaves primarias.
4. Relaciones.
5. Tipos de datos sugeridos.
6. Operaciones necesarias.
7. Módulo que se encargará de la persistencia.
8. Requerimientos relacionados.

Mantén el diseño simple y adecuado para un proyecto académico. No generes código todavía.
```

Prompt para proyectos sin base de datos

```
Actúa como arquitecto de software.

Nuestro proyecto no parece requerir base de datos.

Analiza si esta decisión es correcta.

Proyecto:
```

```
[DESCRIPCIÓN DEL PROYECTO]

Requerimientos:
[PEGAR REQUERIMIENTOS]

Responde:
1. Si realmente podemos trabajar sin base de datos.
2. Qué alternativa de almacenamiento conviene usar.
3. Qué datos serán temporales.
4. Qué datos se guardarán en archivos, si aplica.
5. Qué módulo manejará esos archivos o salidas.
6. Qué riesgos existen al no usar base de datos.

No generes código.
```

7. Bitácora de uso de IA

Fase	Prompt usado	Resultado de IA	Qué aceptamos	Qué modificamos	Responsable
Decisión de persistencia					
Diseño de BD o alternativa					
Plan del módulo					
Implementación					
Revisión					
Pruebas					

8. Preguntas de cierre

1. ¿Qué módulo implementaron primero y por qué?
2. ¿Qué requerimiento atiende ese módulo?
3. ¿El proyecto requiere base de datos o no? ¿Por qué?
4. ¿Qué módulo será responsable de manejar los datos?
5. ¿Qué parte del código generó Codex?
6. ¿Qué revisó o corrigió el equipo?
7. ¿Qué prueba manual demuestra que el módulo funciona?
8. ¿Qué quedó pendiente para la siguiente semana?

9. Tarea para la siguiente semana

1. Corregir el primer módulo implementado.
2. Actualizar la decisión de persistencia si encontraron problemas.
3. Completar el diseño mínimo de base de datos o alternativa de almacenamiento.
4. Seleccionar el segundo módulo a implementar.
5. Agregar evidencia de commits.
6. Actualizar la bitácora de Codex.
7. Preparar dudas técnicas para Codex relacionadas con pruebas, refactorización o integración.